RECURSO DE PROFUNDIZACIÓN DEL TEMA 16

Taller: preparar, verificar y defender el proyecto final

1. Resultado esperado

Creará un expediente final coherente con una versión del código, pruebas de CRUD y autenticación, documentación técnica, manual de usuario, guion de defensa y matriz de retroalimentación.

2. Estructura de entrega

```
entrega-final/
├── codigo/
├── docs/
│   ├── README.md
│   ├── ARQUITECTURA.md
│   ├── API.md
│   ├── PRUEBAS.md
│   ├── MANUAL_USUARIO.md
│   └── LECCIONES_APRENDIDAS.md
├── evidencias/
│   ├── despliegue/
│   ├── pruebas/
│   └── monitoreo/
├── scripts/
│   └── verificar-entrega.js
└── DEFENSA.md
```

3. Ejercicio 1: inventario y versión

Cree una etiqueta de versión, registre commit, fecha, URL del cliente, URL de la API y health check. Compruebe que las capturas y documentos correspondan a esa versión.

```
git status
git log -1 --oneline
git tag -a v1.0.0 -m "Entrega final"
git push origin v1.0.0
```

4. Ejercicio 2: verificar CRUD y autenticación

Cree scripts/verificar-entrega.js. Use cuentas de demostración y no escriba credenciales reales en el repositorio.

```
const api = process.env.API_URL;
const correo = process.env.DEMO_EMAIL;
const password = process.env.DEMO_PASSWORD;

async function request(path, options = {}) {
  const response = await fetch(`${api}${path}`, options);
  const data = response.status === 204 ? null : await response.json();
  if (!response.ok) throw new Error(`${path}: ${response.status} ${JSON.stringify(data)}`);
  return { response, data };
}
```

```
async function run() {
  const login = await request("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ correo, password })
  });
  const headers = {
    "Content-Type": "application/json",
    Authorization: `Bearer ${login.data.token}`
  };

  const created = await request("/api/incidentes", {
    method: "POST",
    headers,
    body: JSON.stringify({
      titulo: "Incidente de demostración final",
      descripcion: "Registro creado por la verificación automatizada.",
      prioridad: "MEDIA"
    })
  });

  const id = created.data.id;
  await request(`/api/incidentes/${id}`, { headers });
  await request(`/api/incidentes/${id}`, {
    method: "PUT",
    headers,
    body: JSON.stringify({
      titulo: "Incidente final actualizado",
      descripcion: "Actualización comprobada correctamente.",
      prioridad: "ALTA"
    })
  });
  await request(`/api/incidentes/${id}`, { method: "DELETE", headers });
  console.log("CRUD y autenticación verificados");
}

run().catch((error) => {
  console.error(error.message);
  process.exit(1);
});


$env:API_URL='https://api.example'
$env:DEMO_EMAIL='evaluador@example.com'
$env:DEMO_PASSWORD='valor-temporal'
node scripts/verificar-entrega.js
```

Añada casos negativos: login incorrecto, solicitud sin token, rol insuficiente y datos inválidos. Registre código esperado, resultado real y captura.

5. Ejercicio 3: documentación

README.md debe incluir propósito, requisitos, instalación, variables, comandos, estructura, pruebas y despliegue. ARQUITECTURA.md explica capas y decisiones. API.md contiene rutas, permisos, cuerpos, respuestas y errores.

MANUAL_USUARIO.md utiliza lenguaje orientado a tareas. Para cada recorrido incluya objetivo, precondición, pasos, resultado y solución de problemas. PRUEBAS.md presenta matriz y enlaces a evidencias.

```
# Ejemplo de caso en PRUEBAS.md
## PF-01 Crear incidente
- Precondición: usuario ANALISTA autenticado.
- Datos: título, descripción y prioridad válidos.
- Pasos: abrir formulario, completar y guardar.
- Esperado: respuesta 201 y registro visible.
- Real: aprobado.
- Evidencia: evidencias/pruebas/PF-01.png
```

6. Ejercicio 4: preparar la defensa

Cree DEFENSA.md con un guion de 10 minutos: problema, usuarios, arquitectura, demostración, seguridad, pruebas, despliegue, limitaciones y cierre. Asigne tiempos y responsable si el trabajo es grupal.

Tabla 1. Guion sugerido para la defensa

Momento	Tiempo	Evidencia
Problema y alcance	1 minuto	Necesidad resuelta
Arquitectura	1 minuto	Diagrama
Demostración	4 minutos	Login y CRUD
Calidad y operación	2 minutos	Pruebas, logs y health
Límites y cierre	2 minutos	Lecciones y mejoras

Ensaye con la URL productiva y prepare un video o capturas como contingencia. Formule respuestas para: ¿cómo se protegen rutas?, ¿qué ocurre si falla la base?, ¿cómo se revierte un despliegue?, ¿qué prueba ofrece mayor confianza?

7. Ejercicio 5: retroalimentación y cierre

Registre cada observación con fuente, tipo, impacto, esfuerzo y decisión. Use estados: aplicar ahora, planificar, investigar o no aplicar. Toda decisión debe incluir una razón.

```
| Observación    | Tipo     |Impacto| Esfuerzo | Decisión      | Acción           |
|----------------|--------  |-------|---------:|---------------|------------------|
| Mejorar filtro | Mejora   | Medio | Bajo     | Aplicar       | Próxima versión  |
| Token visible  | Defecto  | Alto  | Medio    | Aplicar ahora | Corregir y rotar |
```

Complete LECCIONES_APRENDIDAS.md con tres preguntas: ¿qué funcionó?, ¿qué dificultó el trabajo?, ¿qué haríamos diferente? Use ejemplos concretos y relacione cada lección con una acción futura.